```
#include <iostream>
#include <string>
#include <limits>
using namespace std;
// Constants
const int MAX_NODES = 20;
const int INF = 999999;
// Safe integer input function
int readInt(string message) {
    int value;
    while (true) {
        cout << message;
        if (cin >> value) {
            return value;
        }
        cout << "Invalid input! Please enter an integer.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}
// Graph Class
class RoadGraph {
```

```
private:
    int adjMatrix[MAX_NODES][MAX_NODES];
    string locationNames[MAX_NODES];
    int vertexCount;
public:
    // Constructor
    RoadGraph() {
        vertexCount = 0;
        for (int i = 0; i < MAX_NODES; i++) {
            locationNames[i] = "";
            for (int j = 0; j < MAX_NODES; j++) {
                adjMatrix[i][j] = (i == j) ? 0 : INF;
            }
        }
    }
    // 1. Add Location
    void addLocation(string name) {
        if (vertexCount >= MAX_NODES) {
            cout << "Maximum location limit reached!\n";
            return;
        }
        if (name.empty()) {
            cout << "Location name cannot be empty!\n";
```

```
        return;
    }
    if (searchLocation(name) != -1) {
        cout << "Location already exists!\n";
        return;
    }
    locationNames[vertexCount] = name;
    cout << "Location '" << name
         << "' added successfully at index "
         << vertexCount << ".\n";
    vertexCount++;
}
// 2. Add Road
void addRoad(int u, int v, int weight) {
    if (u < 0 || u >= vertexCount ||
        v < 0 || v >= vertexCount) {
        cout << "Invalid location indices!\n";
        return;
    }
    if (u == v) {
        cout << "A location cannot connect to itself!\n";
        return;
    }
```

```
  if (weight <= 0) {
        cout << "Road distance must be positive!\n";
        return;
    }
    adjMatrix[u][v] = weight;
    adjMatrix[v][u] = weight;
    cout << "Road added successfully between "
         << locationNames[u] << " and "
         << locationNames[v] << ".\n";
}
// 3. Delete Road
void deleteRoad(int u, int v) {
    if (u < 0 || u >= vertexCount ||
        v < 0 || v >= vertexCount) {
        cout << "Invalid location indices!\n";
        return;
    }
    if (u == v || adjMatrix[u][v] == INF) {
        cout << "Road does not exist!\n";
        return;
    }
    adjMatrix[u][v] = INF;
    adjMatrix[v][u] = INF;
```

```
    cout << "Road between "
         << locationNames[u] << " and "
         << locationNames[v]
         << " deleted successfully!\n";
}
// 4. Search Location
int searchLocation(string name) {
    for (int i = 0; i < vertexCount; i++) {
        if (locationNames[i] == name) {
            return i;
        }
    }
    return -1;
}
// 5. Dijkstra's Shortest Path Algorithm
void findShortestPath(int start, int dest) {
    if (start < 0 || start >= vertexCount ||
        dest < 0 || dest >= vertexCount) {
        cout << "Invalid location indices!\n";
        return;
    }
    int dist[MAX_NODES];
    int parent[MAX_NODES];
```

```
bool visited[MAX_NODES];
for (int i = 0; i < vertexCount; i++) {
    dist[i] = INF;
    parent[i] = -1;
    visited[i] = false;
}
dist[start] = 0;
for (int count = 0; count < vertexCount; count++) {
    int u = -1;
    int minDist = INF;
    // Find minimum distance unvisited vertex
    for (int i = 0; i < vertexCount; i++) {
        if (!visited[i] && dist[i] < minDist) {
            minDist = dist[i];
            u = i;
        }
    }
    if (u == -1) {
        break;
    }
    visited[u] = true;
    // Update adjacent vertices
    for (int v = 0; v < vertexCount; v++) {
```

```
        if (!visited[v] &&
            adjMatrix[u][v] != INF &&
            adjMatrix[u][v] > 0 &&
            dist[u] + adjMatrix[u][v] < dist[v]) {
            dist[v] = dist[u] + adjMatrix[u][v];
            parent[v] = u;
        }
    }
}
// Check whether path exists
if (dist[dest] == INF) {
    cout << "No path exists between "
         << locationNames[start] << " and "
         << locationNames[dest] << "!\n";
    return;
}
// Reconstruct shortest path
int path[MAX_NODES];
int pathLength = 0;
int current = dest;
while (current != -1 && pathLength < MAX_NODES) {
    path[pathLength++] = current;
    current = parent[current];
```

```
    }
    cout << "\n========== SHORTEST PATH RESULT ==========\n";
    cout << "Shortest Path: ";
    for (int i = pathLength - 1; i >= 0; i--) {
        cout << locationNames[path[i]];
        if (i > 0) {
            cout << " -> ";
        }
    }
    cout << "\nTotal Distance: "
         << dist[dest] << " units.\n";
    cout << "==========================================\n";
}
// 6. Breadth First Search (BFS)
void bfsTraversal(int startNode) {
    if (startNode < 0 || startNode >= vertexCount) {
        cout << "Invalid starting location index!\n";
        return;
    }
    bool visited[MAX_NODES] = {false};
    // Manual array-based queue
    int queue[MAX_NODES];
    int front = 0;
```

```
    int rear = 0;
    visited[startNode] = true;
    queue[rear++] = startNode;
    cout << "\n[BFS Traversal]: ";
    while (front < rear) {
        int curr = queue[front++];
        cout << locationNames[curr] << " ";
        for (int i = 0; i < vertexCount; i++) {
            if (adjMatrix[curr][i] != INF &&
                adjMatrix[curr][i] > 0 &&
                !visited[i]) {
                visited[i] = true;
                queue[rear++] = i;
            }
        }
    }
    cout << endl;
}
// Helper Function for DFS
void dfsUtil(int curr, bool visited[]) {
    visited[curr] = true;
    cout << locationNames[curr] << " ";
    for (int i = 0; i < vertexCount; i++) {
```

```
        if (adjMatrix[curr][i] != INF &&
            adjMatrix[curr][i] > 0 &&
            !visited[i]) {
            dfsUtil(i, visited);
        }
    }
}
// 7. Depth First Search (DFS)
void dfsTraversal(int startNode) {
    if (startNode < 0 || startNode >= vertexCount) {
        cout << "Invalid starting location index!\n";
        return;
    }
    bool visited[MAX_NODES] = {false};
    cout << "\n[DFS Traversal]: ";
    dfsUtil(startNode, visited);
    cout << endl;
}
// 8. Bubble Sort for Nearby Roads
void sortRoadsFromLocation(int nodeIdx) {
    if (nodeIdx < 0 || nodeIdx >= vertexCount) {
        cout << "Invalid location index!\n";
        return;
```

```
}

int weights[MAX_NODES];

int targets[MAX_NODES];

int count = 0;


// Collect connected roads

for (int i = 0; i < vertexCount; i++) {

   if (adjMatrix[nodeIdx][i] != INF &&

       adjMatrix[nodeIdx][i] > 0) {

       weights[count] = adjMatrix[nodeIdx][i];

       targets[count] = i;

       count++;

   }

}

// Bubble Sort

for (int i = 0; i < count - 1; i++) {

   for (int j = 0; j < count - i - 1; j++) {

       if (weights[j] > weights[j + 1]) {

           int tempW = weights[j];

           weights[j] = weights[j + 1];

           weights[j + 1] = tempW;

           int tempT = targets[j];

           targets[j] = targets[j + 1];
```

```
            targets[j + 1] = tempT;
          }
        }
    }
    cout << "\nSorted Roads from "
         << locationNames[nodeIdx]
         << " (Nearest to Farthest):\n";
    if (count == 0) {
        cout << "No connected roads found!\n";
        return;
    }
    for (int i = 0; i < count; i++) {
        cout << " -> "
             << locationNames[targets[i]]
             << " (Distance: "
             << weights[i] << ")\n";
    }
}
// 9. Display Graph using Operator Overloading
friend ostream& operator<<(ostream& out, const RoadGraph& g) {
    out << "\n=======================================\n";
    out << "       SMART ROAD NETWORK MAP          \n";
    out << "=======================================\n";
```

```
out << "Total Registered Locations: "
   << g.vertexCount << "\n\n";
out << "Location Index Mapping:\n";
for (int i = 0; i < g.vertexCount; i++) {
   out << "[" << i << "] -> "
       << g.locationNames[i] << "\n";
}
out << "\nAdjacency Matrix (Distances):\n\t";
for (int i = 0; i < g.vertexCount; i++) {
   out << "[" << i << "]\t";
}
out << "\n";
for (int i = 0; i < g.vertexCount; i++) {
   out << "[" << i << "]\t";
   for (int j = 0; j < g.vertexCount; j++) {
       if (g.adjMatrix[i][j] == INF) {
           out << "INF\t";
       }
       else {
           out << g.adjMatrix[i][j] << "\t";
       }
   }
```

```
            out << "\n";
        }
        out << "========================================\n";
        return out;
    }
    // Get Number of Locations
    int getVertexCount() const {
        return vertexCount;
    }
};
// MAIN FUNCTION
int main() {
    RoadGraph map;
    int choice;
    do {
        cout << "\n======================================\n";
        cout << "   SMART ROAD NETWORK & ROUTE FINDER    \n";
        cout << "======================================\n";
        cout << "1. Add Location (Vertex)\n";
        cout << "2. Add Road (Weighted Edge)\n";
        cout << "3. Display Road Network\n";
        cout << "4. Find Shortest Path (Dijkstra)\n";
        cout << "5. Graph Traversal (BFS)\n";
```

```
cout << "6. Graph Traversal (DFS)\n";
cout << "7. Search Location & Sort Roads\n";
cout << "8. Delete Road\n";
cout << "9. Exit\n";
choice = readInt("Enter your choice: ");
switch (choice) {
   // Add Location
   case 1: {
      string name;
      cout << "Enter location name: ";
      cin >> name;
      map.addLocation(name);
      break;
   }
   // Add Road
   case 2: {
      if (map.getVertexCount() < 2) {
         cout << "Add at least 2 locations first!\n";
         break;
      }
      int u, v, weight;
      u = readInt("Enter source location index: ");
      v = readInt("Enter destination location index: ");
```

```
    weight = readInt("Enter road distance: ");
    map.addRoad(u, v, weight);
    break;
}
// Display Graph
case 3: {
    cout << map;
    break;
}
// Shortest Path
case 4: {
    if (map.getVertexCount() < 2) {
        cout << "Not enough locations!\n";
        break;
    }
    int start, dest;
    start = readInt("Enter starting location index: ");
    dest = readInt("Enter destination location index: ");
    map.findShortestPath(start, dest);
    break;
}
// BFS
case 5: {
```

```
    if (map.getVertexCount() == 0) {
        cout << "Road network is empty!\n";
        break;
    }
    int start;
    start = readInt("Enter starting location index: ");
    map.bfsTraversal(start);
    break;
}
// DFS
case 6: {
    if (map.getVertexCount() == 0) {
        cout << "Road network is empty!\n";
        break;
    }
    int start;
    start = readInt("Enter starting location index: ");
    map.dfsTraversal(start);
    break;
}
// Search and Sort
case 7: {
```

```
    string query;
    cout << "Enter location name to search: ";
    cin >> query;
    int idx = map.searchLocation(query);
    if (idx != -1) {
        cout << "Location found at index: "
             << idx << "\n";
        map.sortRoadsFromLocation(idx);
    }
    else {
        cout << "Location not found!\n";
    }
    break;
}
// Delete Road
case 8: {
    if (map.getVertexCount() < 2) {
        cout << "Not enough locations!\n";
        break;
    }
    int u, v;
    u = readInt("Enter source location index: ");
    v = readInt("Enter destination location index: ");
```

```
            map.deleteRoad(u, v);
            break;
        }
        // Exit
        case 9: {
            cout << "Thank you for using Smart Road Network!\n";
            break;
        }
        default: {
            cout << "Invalid choice! Select between 1 and 9.\n";
        }
      }
   } while (choice != 9);
   return 0;
}
```

OUTPUT:

```
========================================
  SMART ROAD NETWORK & ROUTE FINDER
========================================
1. Add Location (Vertex)
2. Add Road (Weighted Edge)
```

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 1

Enter location name: library

Location 'library' added successfully at index 0.

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 1

Enter location name: canteen

Location 'canteen' added successfully at index 1.

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 1

Enter location name: principal office

Location 'principal' added successfully at index 2.

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: Invalid input! Please enter an integer.

Enter your choice: 2

Enter source location index: 0

Enter destination location index: 1

Enter road distance: 10

Road added successfully between library and canteen.

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 2

Enter source location index: 1

Enter destination location index: 2

Enter road distance: 15

Road added successfully between canteen and principal.

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 3

==========

SMART ROAD NETWORK MAP

==========

Total Registered Locations: 3

Location Index Mapping:

[0] -> library

[1] -> canteen

[2] -> principal

Adjacency Matrix (Distances):

	[0]	[1]	[2]
[0]	0	10	INF
[1]	10	0	15
[2]	INF	15	0

==========

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 4

Enter starting location index: 1

Enter destination location index: 2

========== SHORTEST PATH RESULT ==========

Shortest Path: canteen -> principal

Total Distance: 15 units.

==========

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 5

Enter starting location index: 0

[BFS Traversal]: library canteen principal

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 1

Enter location name: hostel

Location 'hostel' added successfully at index 3.

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 6

Enter starting location index: 2

[DFS Traversal]: principal canteen library

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 7

Enter location name to search: canteen

Location found at index: 1

Sorted Roads from canteen (Nearest to Farthest):

-> library (Distance: 10)

-> principal (Distance: 15)

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 8

Enter source location index: 0

Enter destination location index: 1

Road between library and canteen deleted successfully!

==========

SMART ROAD NETWORK & ROUTE FINDER

==========

1. Add Location (Vertex)

2. Add Road (Weighted Edge)

3. Display Road Network

4. Find Shortest Path (Dijkstra)

5. Graph Traversal (BFS)

6. Graph Traversal (DFS)

7. Search Location & Sort Roads

8. Delete Road

9. Exit

Enter your choice: 9

Thank you for using Smart Road Network!